# Homework 10: SQL **hw10.zip (hw10.zip)**

*Due by 11:59pm on Thursday, December 5*

## Instructions

Download hw10.zip (hw10.zip).

**Submission:** When you are done, submit the assignment by uploading all code files you've edited to Gradescope. You may submit more than once before the deadline; only the final submission will be scored. Check that you have successfully submitted your code on Gradescope. See Lab 0 (../../lab/lab00#task-c-submitting-the-assignment) for more instructions on submitting assignments.

**Using Ok:** If you have any questions about using Ok, please refer to this guide. (../../articles/using-ok)

**Grading:** Homework is graded based on correctness. Each incorrect problem will decrease the total score by one point. **This homework is out of 2 points.**

To check your progress, you can run `sqlite3` directly by running:

```
python3 sqlite_shell.py --init hw10.sql
```

You should also check your work using `ok`:

```
python3 ok
```

## Visualizing SQL

If you would like some support with visualizing SQL, please navigate to code.cs61a.org (https://code.cs61a.org/) and select *Start SQL Interpreter*.

From there, you can either practice on tables you make yourself, existing tables from the CS 61A database (which can be found by typing `.tables`), or on the tables from the assignment you are working on by copying and pasting the entire `CREATE TABLE ...` command.

# Required Questions

Getting Started Videos

# SQL

## Dog Data

In each question below, you will define a new table based on the following tables.

```
CREATE TABLE parents AS
  SELECT "ace" AS parent, "bella" AS child UNION
  SELECT "ace"          , "charlie"        UNION
  SELECT "daisy"        , "hank"           UNION
  SELECT "finn"         , "ace"            UNION
  SELECT "finn"         , "daisy"          UNION
  SELECT "finn"         , "ginger"         UNION
  SELECT "ellie"        , "finn";

CREATE TABLE dogs AS
  SELECT "ace" AS name, "long" AS fur, 26 AS height UNION
  SELECT "bella"      , "short"      , 52           UNION
  SELECT "charlie"    , "long"       , 47           UNION
  SELECT "daisy"      , "long"       , 46           UNION
  SELECT "ellie"      , "short"      , 35           UNION
  SELECT "finn"       , "curly"      , 32           UNION
  SELECT "ginger"     , "short"      , 28           UNION
  SELECT "hank"       , "curly"      , 31;

CREATE TABLE sizes AS
  SELECT "toy" AS size, 24 AS min, 28 AS max UNION
  SELECT "mini"       , 28       , 35        UNION
  SELECT "medium"     , 35       , 45        UNION
  SELECT "standard"   , 45       , 60;
```

Your tables should still perform correctly *even if the values in these tables change*. For example, if you are asked to list all dogs with a name that starts with `h`, you should write:

```
SELECT name FROM dogs WHERE "h" <= name AND name < "i";
```

In other words, you should not assume that the `dogs` table has only the data in the table above by writing:

```
SELECT "hank";
```

The former query would still be correct if the name `ginger` were changed to `gigi` or a row was added with the name `harry`. Contrastingly, writing `SELECT "hank";` would not.

## Q1: By Parent Height

Create a table `by_parent_height` that has a column of the names of all dogs that have a `parent`, ordered by the height of the parent dog from tallest parent to shortest parent.

```
-- All dogs with parents ordered by decreasing height of their parent
CREATE TABLE by_parent_height AS
  SELECT "REPLACE THIS LINE WITH YOUR SOLUTION";
```

For example, `finn` has a parent `ellie` with height 35, and so should appear before `ginger` who has a parent `finn` with height 32. The names of dogs with parents of the same height should appear together in any order. For example, `bella` and `charlie` should both appear at the end, but either one can come before the other.

The `by_parent_height` table should look like this:

```
+----------+
| chil     |
+----------+
| hank     |
| finn     |
| ace      |
| daisy    |
| ginger   |
| bella    |
| charlie  |
+----------+
```

Use Ok to test your code:

```
python3 ok -q by_parent_height
```

## Q2: Size of Dogs

The Fédération Cynologique Internationale classifies a standard poodle as over 45 cm and up to 60 cm. The `sizes` table describes this and other such classifications, where a dog must be over the `min` and less than or equal to the `max` in `height` to qualify as `size`.

Create a `size_of_dogs` table with two columns, one for each dog's `name` and another for its `size`.

```
-- The size of each dog
CREATE TABLE size_of_dogs AS
  SELECT "REPLACE THIS LINE WITH YOUR SOLUTION";
```

The `size_of_dogs` table should look like this:

```
+----------+----------+
| name     | size     |
+----------+----------+
| ace      | toy      |
| bella    | standard |
| charlie  | standard |
| daisy    | standard |
| ellie    | mini     |
| finn     | mini     |
| ginger   | toy      |
| hank     | mini     |
+----------+----------+
```

Use Ok to test your code:

```
python3 ok -q size_of_dogs
```

## Q3: Sentences

There are two pairs of siblings that have the same size. Create a table that contains a row with a string for each of these pairs. Each string should be a sentence describing the siblings by their size.

```
-- [Optional] Filling out this helper table is recommended
CREATE TABLE siblings AS
  SELECT "REPLACE THIS LINE WITH YOUR SOLUTION";

-- Sentences about siblings that are the same size
CREATE TABLE sentences AS
  SELECT "REPLACE THIS LINE WITH YOUR SOLUTION";
```

Each sibling pair should appear only once in the output, and siblings should be listed in alphabetical order (e.g. `"bella and charlie..."` instead of `"charlie and bella..."`), as follows:

```
sqlite> SELECT * FROM sentences;
The two siblings, bella and charlie, have the same size: standard
The two siblings, ace and ginger, have the same size: toy
```

**Hint**: First, create a helper table containing each pair of siblings. This will make comparing the sizes of siblings when constructing the main table easier.

**Hint**: If you join a table with itself, use `AS` within the `FROM` clause to give each table an alias.

**Hint**: In order to concatenate two strings into one, use the `||` operator, e.g. `SELECT "hello" || "world";` will return `helloworld`.

Use Ok to test your code:

```
python3 ok -q sentences
```

## Q4: Low Variance

We want to create a table that contains the *height range* (defined as the difference between maximum and minimum `height`) of all dogs that share a `fur` type. However, we'll only consider `fur` types where each dog with that `fur` type is within 30% of the average `height` of all dogs with that `fur` type; we call this the *low variance* criterion.

For example, if the average `height` for short-haired dogs is 10, then in order to be included in our output, all dogs with short hair must have a `height` of at most 13 and at least 7 (*inclusive*).

**Hint**: `MIN`, `MAX`, and `AVG` will be useful here. **Hint**: You may want to first find the average height and make sure that:

```
* There are no heights smaller than 0.7 (i.e. 70%) of the average.
* There are no heights greater than 1.3 (i.e. 130%) of the average.
```

```
-- Height range for each fur type where all of the heights differ by no more than 30% f
CREATE TABLE low_variance AS
  SELECT "REPLACE THIS LINE WITH YOUR SOLUTION";
```

Your output should have two columns, in this order: the `fur` type and the `height_range` for the `fur` types that meet this criteria. It should look like this:

```
+------------+--------------+
| fur        | height_range |
+------------+--------------+
| Curly      | 1            |
+------------+--------------+
```

Explanation

The average height of long-haired dogs is 39.7, so the low variance criterion requires the height of each long-haired dog to be between 27.8 and 51.6. However, `ace` is a long-haired dog with height 26, which is outside this range. For short-haired dogs, `bella` falls outside the valid range (check!). Thus, neither short nor long haired dogs are included in the output. There are two curly haired dogs: `finn` with height 32 and `hank` with height 31. This gives a height range of 1.

Use Ok to test your code:

```
python3 ok -q low_variance
```

# Submit Assignment

Submit this assignment by uploading any files you've edited **to the appropriate Gradescope assignment.** Lab 00 (../../lab/lab00/#submit-with-gradescope) has detailed instructions.

Make sure to submit `hw10.sql` to the autograder!

# Exam Practice

The following are some SQL exam problems from previous semesters that you may find useful as additional exam practice.

1. Fall 2019 Final, Question 10: Big Game (https://cs61a.org/exam/fa19/final/61a-fa19-final.pdf#page=11)
2. Summer 2019 Final, Question 8: The Big SQL (https://cs61a.org/exam/su19/final/61a-su19-final.pdf#page=11)
3. Fall 2018 Final, Question 7: SQL of Course (https://inst.eecs.berkeley.edu/~cs61a/fa18/assets/pdfs/61a-fa18-final.pdf#page=9)